

Levine–Gates assignment model — desk reference

Last synced: 2026-09-24

Audience: Charles Levine — printable quick reference while continuing to understand and develop the model.

Standalone: This document explains why the Levine–Gates model was built, how it fits the three-step generative pipeline (assign → score → select), and every main formula with plain-English meaning. No other file is required to use it.

Print to PDF (from repo root):

```
./scripts/convert_single_md_to_pdf.sh 3-Master_Plan/re_entry/LG_model_desk_reference.md
```

Binding rule (project lock): Peer **environment** is not the same as **advancement**. Advancement is **score**, then **select**. The Hero curve is an empirical **outcome**; it is not the scoring equation. See `BINDING_Selection_is_its_own_step.md`.

Name note: **Levine–Gates** (abbreviated **LG** on some slides) is the assignment simulation built in August 2026. Repository code paths still use the internal label `grandchild_*` from the Parent / Child / Grandchild prototype family.

Part I — What problem the model solves

The dissertation question (one paragraph)

You observe **sorting**: players (or officers, or scholars) end up grouped with peers of similar measured ability. You also observe a **Hero pattern**: the probability of a scarce positive outcome (draft, promotion, tenure) can rise with peer quality up to a point, then fall at the very top — an inverted-U when outcome is plotted against leave-one-out peer context.

Those are **outcomes**. They do not, by themselves, tell you the **generative rules** that produced rosters and then picked winners. The Levine–Gates work is one piece of a larger program: build a **minimal, domain-general assignment mechanism**, calibrate it to empirical sorting, then layer **scoring** and **selection** on top so you can test whether congestion in the score changes who wins under a fixed winner rule.

LG as a network model in its own right

LG also serves as a **bipartite network generator with prescribed degrees and metadata-based homophily**. One node set contains individuals and the other contains groups. In the implemented model, each individual has degree one and each group’s final degree equals its prescribed roster capacity, with total capacity equal to the number of individuals.

Degrees provide the constraints; metadata provides the preference. The current preference kernel compares scalar individual ability with the group’s evolving mean ability. Group composition emerges through sequential seating and immediate centroid updates. At zero homophily, seating reduces to uniform matching over remaining group-side stubs; positive homophily biases that process by metadata similarity. The code does not yet implement arbitrary individual degrees or arbitrary multivariate metadata: those extensions would require their own specification.

This independent assignment model can be studied before adding scoring or scarce-outcome selection. Its use inside the dissertation pipeline is a second, complementary role.

Three jobs — do not merge them

Job	Question	Main objects
Assign	Who sits with whom in a capacity-limited pool?	Homophily parameter ρ (rho), roster capacities, team centroids
Score	How do we rank candidates for the scarce slot?	Selection score S_i , congestion weight λ (lambda), team congestion L_C
Select	Given ranks, who wins ?	Slot count K , indicator Y_{selected}

The **Hero plot** (draft rate vs leave-one-out pool quality on real data) lives in a fourth layer: **empirical outcomes**. It informs calibration and validation; it is **not** the formula for S_i .

Why assignment needed its own model

Before August 2026, the simulation used **soft assignment**: each synthetic team drew an exogenous target talent level, and players probabilistically matched to those fixed targets. That was useful for teaching and for early knockouts, but it **injected between-team heterogeneity from outside the seating process**. Real NCAA rosters, by contrast, are the result of many sequential choices under roster limits — and empirical team means **emerge** from who actually landed together.

The open scientific question was:

*Can **initially identical** teams **differentiate endogenously** when each arriving individual prefers teams whose **current** membership is similar to their own ability, subject to **hard roster capacity**?*

Here “initially identical” describes the original equal-capacity prototype: teams shared both their initial centroid and capacity. In empirical-capacity runs, **initial centroids remain identical, but capacities may differ**; initial team probabilities then differ in proportion to their open seats.

If yes, you have a credible **assign** layer that can be calibrated to empirical **sorting** before you ask score-and-select questions.

Part II — How the model family developed (Parent → Child → Grandchild)

The August 2026 prototype deliberately kept three related models on paper. Only the **Grandchild** was coded as the working **Levine–Gates** engine; Parent and Child remain as design context.

Shared ingredients (all three models)

- 1. **Bipartite assignment**: individuals on one side, teams on the other; each individual gets exactly one team.
- 2. **Stub capacity**: team j has R_j remaining open seats at each step. When $R_j = 0$, that team receives zero assignment probability. Lineage: configuration-model stub matching (Fosdick et al., 2018).
- 3. **Exponential homophily kernel**: attraction decays with absolute ability distance $|A_i - \mu_j|$. Lineage: Quayle, Siddiqui & Jones (2006). The **full weighted probability** below is **our synthesis** — not a single quoted result from either source.

Parent — dynamic, lagged centroids (future-facing)

Teams have centroids $\mu_{j,t-1}$ frozen **during** period t . After the period completes, centroids update for $t + 1$. This is the natural home for turnover, tenure clocks, and reassignment — **not** implemented in the current basketball prototype.

Child — one-shot, fixed empirical centroids

Each team keeps a **fixed** centroid μ_j (for example the empirical team mean from NCAA data). Players arrive in random order; one complete pass assigns everyone. **One formation run = one stochastic realization**. Repeated runs reset everything; they are **not** longitudinal periods. **Limitation that motivated the Grandchild**: fixed empirical centroids **pre-load** between-team differences. They may not generate enough **endogenous** sorting from the seating rule alone.

Grandchild → Levine–Gates model (what we code)

Same one-shot, hard-capacity, random-order seating as the Child, but:

- Every team starts **empty** with the **same** initial centroid $\mu_j^*(0) = \mu_0 = \hat{A}$ (league mean ability).
 - After each seat, the **receiving team’s centroid updates immediately** from **actual members only**.
 - The **first** player on an empty team sets $\mu_j = A_i$ exactly. The league mean μ_0 is a starting signal, **not** a pseudo-player counted in the roster.
- Why this variant**: differences in final team ability composition emerge through (1) stochastic early assignments, (2) path-dependent centroid updates, (3) similarity-biased later assignments, and (4) finite capacity, rather than being supplied as fixed empirical team means. With heterogeneous capacities, group-size differences are supplied as inputs; only the initial centroids are identical.

Part III — Step 1: Assign (Levine–Gates seating)

Narrative (read once)

Fix a player pool of size N with abilities $\hat{A}_1, \dots, \hat{A}_N$. Fix roster capacities C_j (open seats on team j at start). Permute players uniformly at random. For each player i in that order:

1. Consider every team j with $R_j > 0$.
2. Compute an unnormalized weight $\tilde{w}_{ij}$.
3. Normalize to probabilities $P(j | i)$ and **sample** a team.
4. Decrement R_j , update team j ’s centroid from its members, proceed to the next player.

When all players are seated, you have a **partition** $g(i)$ = team index of player i , and final team means $\mu_j = \bar{T}_j$.

Unnormalized assignment weight

For player i considering team j with $R_j > 0$ open seats:

$$\tilde{w}_{ij} = R_j \exp(-\rho |\hat{A}_i - \mu_j|)$$

Normalized seat probability

$$P(j | i) = \frac{\tilde{w}_{ij}}{\sum_{j'} \tilde{w}_{ij'}}$$

Teams with $R_j = 0$ are excluded from the sum. At $p = 0$:

$$P(j | i) = \frac{R_j}{\sum_{j'} R_{j'}}$$

— assignment is **uniform over remaining open seats** (configuration-model stubs), **not** uniform over teams. Team j gets probability $R_j / \Sigma_{\{j'\}}$ because it has R_j stubs left. This equals uniform-over-teams only when every team with $R_j > 0$ has the same R_j at that step (for example, early in a league with equal caps before rosters diverge).

Centroid update after player i joins team j

If team j already had n members with centroid μ_j before i arrives:

$$\mu_j \leftarrow \frac{n \mu_j + \hat{A}_i}{n + 1}$$

First member on an empty team: $\mu_j \leftarrow \hat{A}_i$ (not μ_0).

Symbol glossary (assign layer)

Symbol	Plain name	Meaning
$\hat{A}_i$	Estimated ability of player i	In standard runs: within-season z-score of points per minute on the locked hero panel (not a fresh random draw).
μ_j	Team centroid	Running mean ability of players already on team j during seating; final value T_j after assign completes.
$\mu_0 = \hat{A}$	League mean	$(1/N) \sum_i \hat{A}_i$; initial centroid for every empty team only.
R_j	Remaining stub capacity	Open seats on team j; decrements by 1 after each assignment.
ρ	Homophily strength	Assign-only knob. Higher $\rho \rightarrow$ stronger preference for teams whose current centroid is close to $\hat{A}_i$.
C_j	Roster capacity	Initial $R_j^{(0)} = C_j$ (often heterogeneous empirical caps; see below).
$g(i)$	Team index	Player i's team after assign; code column often <code>pool_id</code> .

What is held from NCAA vs what is synthetic (standard panel runs)

For each season $t = 2011, \dots, 2021$ in the main comparability scripts:

Held from data	Synthetic
Ability vector $\hat{A}_i$ (same filtered player-seasons as empirical Hero)	Team labels $g(i)$ — who sits with whom
Per-team roster capacities (exact multiset of NCAA roster sizes)	
Total N_t and draft count K_t for select	

Early sweeps used uniform $C = 15$ for every team ($J = N / 15$ synthetic teams). Production comparability runs prefer **empirical roster caps** so total capacity matches real listings.

Part IV — Assign diagnostics (after seating, before score)

These statistics are computed on a **fixed partition** (real NCAA rosters **or** one Levine–Gates realization). They describe **roster geometry**, not the scoring or selection steps.

Within-team mean squared error D

$$D = \frac{1}{N} \sum_{i=1}^N \left(\hat{A}_i - \mu_{g(i)} \right)^2$$

Question answered: “How far is each player, on average, from their own team’s mean?”
Call this within-team dispersion. Do not call **D** “assortativity.”

Global within-team sum of squares global_wss

$$\text{global_wss} = \sum_{i=1}^N \left(\hat{A}_i - \mu_{g(i)} \right)^2 = N \cdot D$$

Same information as **D**, in raw sum units (grows with **N**).

Sorting index H_sort

$$H_{\text{sort}} = 1 - \frac{\sum_i \left(\hat{A}_i - \mu_{g(i)} \right)^2}{\sum_i \left(\hat{A}_i - \bar{A} \right)^2} = 1 - \frac{\text{global_wss}}{\text{SS}_{\text{total}}}$$

where **SS_total** is total sum of squares around the league mean $\hat{A}$.
Question answered: “If I know which roster a player is on, what fraction of their deviation from the league average is explained by that roster’s mean?”

H_sort	Plain reading
Near 0	Team means explain little ability variance; assess against a random-assignment baseline with the same capacities.
Near 1	Within-team ability variation is small relative to total variation.

Random-assignment baseline: $\rho = 0$ does not force $H_{\text{sort}} = 0$. Random roster composition produces differences in sample team means, so realized explained variance can be positive even without homophilic preference. Interpret H_{sort} relative to random assignments with the same ability vector and capacities; the positive baseline in the sweep below is consistent with this distinction.

Critical distinction:

- $\mathbf{p}$ is the **generative** homophily knob in the assign rule.
- $\mathbf{H_sort}$ is a **realized outcome** on whatever partition you ended up with (NCAA or simulated).

Correct wording: “At $\mathbf{p} = 0.5$, realized $\mathbf{H_sort} \approx 0.15$ on this assign realization.”

Incorrect wording: “We set assortativity to 0.15.”

Illustrative 2015 sweep (uniform $\mathbf{C} = 15$, 30 repetitions per $\mathbf{p}$): $\mathbf{H_sort}$ rises from about **0.07** at $\mathbf{p} = 0$ to about **0.32** at $\mathbf{p} = 1$; $\mathbf{global_wss}$ falls monotonically on the same runs.

Calibrating $\mathbf{p}$ to empirical sorting (post-prototype)

After the engine existed, the calibration task (Paper Directions 21) was:

Find $\mathbf{p}$ such that Levine–Gates simulated $\mathbf{H_sort}$ matches empirical NCAA $\mathbf{H_sort}$ on the same ability panel and filters.

Method in use: bracket search over $\mathbf{p}$, many random seeds per trial, compare mean simulated $\mathbf{H_sort}$ to empirical $\mathbf{H_sort}$. Formal maximum-likelihood roster matching remains parked; matching a small set of summary statistics (sorting index, within-team spread, congestion shape) is the practical approach. Options are documented in `Alex_notes/rho_est_options_for_dummies.md`.

Part V — Step 2: Score (ranking only)

Assign has finished; rosters are fixed. **Score** produces a **selection score** $\mathbf{S_i}$ for every player. λ enters here only — **not** in assign.

Selection score (version 1)

$$S_i = \hat{A}_i - \lambda L_C(i)$$

Symbol	Meaning
$\mathbf{S_i}$	Selection score — higher means better rank for select. Repository column: <code>selection_weight</code> .
$\hat{\mathbf{A}}_i$	Same ability units as assign (typically within-season points-per-minute z-score).
λ	Congestion weight in the score . $\lambda = 0 \rightarrow$ rank by talent only.
$\mathbf{L_C(i)}$	Team congestion for player i ’s team — same value for every teammate on that team.

Soft peer viability (building block)

For any player $\mathbf{k}$ on the roster:

$$v_k = \sigma(\gamma(\hat{A}_k - \theta)) = \frac{1}{1 + \exp(-\gamma(\hat{A}_k - \theta))}$$

Symbol	Meaning
$\sigma(\cdot)$	Logistic function — soft “is this peer competitively viable?”
θ	Viability cutline (center of the logistic).
$\mathbf{v}$	Sharpness: large $\mathbf{v} \approx$ hard step at θ ; small $\mathbf{v} \approx$ smooth ramp.

Standard comparability default: $\mathbf{v} = 0.5$ (characterization work may sweep $\mathbf{v}$ separately).

Team congestion $\mathbf{L_C}$ (Paper Directions 16 lock)

For player i on team $\mathbf{j} = \mathbf{g(i)}$, with roster $\{ \mathbf{k} : \mathbf{g(k)} = \mathbf{j} \}$:

$$L_C(i) = L_{C,j} = \frac{1}{|j|} \sum_{k \in j} v_k = \frac{1}{|j|} \sum_{k \in j} \sigma(\gamma(\hat{A}_k - \theta))$$

Include self in the team mean. Every teammate shares the **same** $\mathbf{L_{\{C,j\}}}$.

Interpretation: Average soft “viable-peer pressure” on the roster — how crowded the team is with draft-competitive talent.

Contrast (visualization only): **Leave-one-out** pool quality `poolq_loo` excludes player i from the teammate mean. That axis bins the empirical Hero plot. It is **not** the congestion term in $\mathbf{S_i}$ when using team $\mathbf{L_C}$ mode.

Setting θ from K and N

Let $F_t(\cdot)$ be the cumulative distribution function of within-season $\hat{A}$ in season t . Let K_t be empirical draft count and N_t the filtered panel size. Then:

$$\theta_t = F_t^{-1}\left(1 - \frac{K_t}{N_t}\right)$$

In words: θ is the ability quantile where the top fraction K/N of the specified pool begins. If $K/N = 1\%$, θ is the 99th percentile. The 1% figure is an illustrative MBB-like scarcity setting, not a verified rate for every saved run. Record each run’s actual K , N , eligibility filters, and outcome definition; a count derived from ever-drafted player-seasons must not be described as an annual draft count without verification.

Per season: K , N , and θ may change year to year.

Unit matching when $\hat{A}$ is z-scored

If $\hat{A}$ is standardized and $L_C \in [0, 1]$, implementations may multiply L_C by a scale derived from the ability spread (for example 90th minus 10th percentile of $\hat{A}$) so that $\lambda \cdot L_C$ can move ranks. Check `L_term_scale` / crowding scale flags in simulation config when reproducing a run.

Part VI — Step 3: Select (winner rule)

Given S_i , select applies a **winner rule**. Version 1 is deterministic:

$$Y_i = \mathbf{1}\{S_i \text{ is among the top } K \text{ scores in the season}\}$$

Symbol	Meaning
K	Number of scarce slots (empirical draft count per season in standard runs).
K/N	Selectivity rate (system feature).
Y_{selected}	1 if selected in the simulation; 0 otherwise.

Future extension (not version 1): stochastic selection drawn from scores — change **select only**, leave S_i fixed.

What to plot after select

- Bin players by leave-one-out pool quality (**poolq_loo**) — same axis family as the empirical Hero.
- y-axis:** fraction with **$Y_{\text{selected}} = 1$** in each bin.
- Compare NCAA (real rosters, real draft) vs Levine–Gates (simulated rosters, top- K on simulated scores).

Historical result — saved output, not independently reproduced here: The August 12, 2026 [sweep metadata](#) records 2011–2021 player-seasons, empirical roster capacities, $\rho = 0.5$, $\gamma = 0.5$, and base seed 5412015. Its leave-one-out shape diagnostic labels the $\lambda = 1.25$ run “monotone increasing” and the tested $\lambda = 1.5, 1.75, 2, 3, 4$ runs “inverted-U-like.”

What those labels establish: the [diagnostic](#) labels a curve inverted-U-like when its highest bin is interior and above both endpoints. Its “monotone increasing” label means the highest bin is the last bin; it does not test every adjacent increase. Neither label is a test of quadratic curvature.

Limits: these are run-specific shape summaries, not a universal congestion threshold or a controlled necessity result. The current [sweep implementation](#) changes the assignment seed across λ arms, so it does not isolate congestion on identical rosters. Numerical λ comparisons also depend on congestion scaling, viability settings, score transformations, and the winner rule. Whether congestion can produce the relevant effect **without assortative preference remains the open PD41 question**; it requires the separately proposed comparison.

Part VII — Full pipeline (one page)

Player pool ($\hat{A}_i$ from data)
↓
ASSIGN – Levine–Gates, homophily ρ , roster caps
↓
Rosters $g(i)$, team means $\hat{t}_j = \mu_j$
↓
SCORE – $S_i = \hat{A}_i - \lambda L_C$ (viability σ , cutline θ , sharpness γ)
↓
SELECT – top K by $S_i \rightarrow Y_{\text{selected}}$
↓
Compare bin-wise selection rate vs Hero (often vs poolq_loo bins)

Step	Direct controls
Assign	ρ , capacities, random seed
Score	λ, θ, γ
Select	K , winner rule type

Distinct steps, potentially coupled settings: K directly controls selection capacity, but under $\theta = F^{-1}(1 - K/N)$ it also changes the viability threshold used in SCORE. Changing K while recomputing θ therefore changes both scoring and selection. To isolate selection scarcity

alone, hold λ , θ , γ , congestion scaling, the assigned rosters, and the winner-rule type fixed while varying K . That is the design proposed in the [PD41 working brief](#), not a change already applied to historical runs.

Part VIII — Objects people confuse (keep separate)

Often confused	Keep separate because
ρ vs λ	ρ = assign homophily; λ = score congestion weight
Score vs select	S_i ranks; top-K picks winners
Team L_C vs leave-one-out pool quality	Team congestion enters S_i ; leave-one-out bins the Hero axis
T_j vs exogenous targets	Levine–Gates μ_j is endogenous ; legacy soft-assign used fixed targets
Hero curve vs S_i	Hero = empirical outcome plot; S_i = generative ranking rule
H_{sort} vs ρ	H_{sort} = realized sorting readout; ρ = assign generative knob
Homophily vs assortativity	Homophily is preference in the assign rule; H_{sort} is realized partition sorting

Part IX — Implementation pointers

Topic	Location
Assign engine	sports/541_grandchild_homophily_assign.py → grandchild_assign
Score / select engine	sports/tier1_pool_assignment.py
Defaults and knobs	sports/tier1_sim_config.py
Method note (what was added Aug 2026)	sports/documents/541_grandchild_homophily_assign_README.md
Prototype genesis (Parent / Child / Grandchild)	3-Master_Plan/COMPASS_work/COMPASS_DETAILED_ASSIGN_GRANDCHILD_INSTRUCTIONS.md
ρ calibration (Paper Directions 21)	3-Master_Plan/re_entry/HER0s_and_PASSes/pd21_rho/README.md
H_{sort} / D deep dive	3-Master_Plan/re_entry/HER0s_and_PASSes/grandchild_assign/GRANDCHILD_D_and_H_sort_interpretation.md
Three-layer dissertation framing	3-Master_Plan/re_entry/02_Three_Kinds_of_Model.md
Sim re-entry contract	sports/540_READ_ME_SIM.md

Part X — Suggested reading order inside this project

If you are re-entering after time away:

1. This desk reference (assign → score → select skeleton).
2. [02_Three_Kinds_of_Model.md](#) — why the layers must stay separate.
3. [COMPASS_DETAILED_ASSIGN_GRANDCHILD_INSTRUCTIONS.md](#) — prototype mission and validation locks.
4. [GRANDCHILD_D_and_H_sort_interpretation.md](#) — when interpreting ρ sweeps and empirical vs simulated partitions.

Revision note — 2026-09-24 (VECTOR, authorized by Charles): Clarified LG’s independent network-model role, equal-centroid versus equal-capacity assumptions, the random-assignment sorting baseline, the coupling between selection capacity and viability, and the provenance and limits of the historical shape claim. The corrected open-seat probability explanation is retained. This revision changes documentation only.

End of desk reference.